

# **Integração e Evolução de Sistemas de Informação**

Arquitetura de Sistemas de Informação

Juscimara G. Avelino

[jga2@cin.ufpe.br](mailto:jga2@cin.ufpe.br)

# O que é uma Arquitetura?

*Different elements working  
smoothly together*

# O que é uma Arquitetura?

- Estrutura dos sistemas de informação
- Elementos e suas relações
- Define o comportamento visível dos componentes

# **Por que estudar Arquitetura de Sistemas?**

# Por que estudar Arquitetura de Sistemas

- Sistemas estão cada vez mais complexos
- Arquitetura define como partes do sistema se relacionam
- Impacta diretamente na manutenção, escalabilidade e evolução

# 'Structure matters' – Bass et al.

- Estrutura é um fator essencial no sucesso de sistemas
- Afeta diretamente robustez, flexibilidade e estabilidade

# Arquitetura intencional vs accidental

- **Intencional:** planejada desde o início do projeto
- **Accidental:** emerge das decisões de design não coordenadas
- Toda aplicação complexa tem uma arquitetura – seja boa ou ruim

# Níveis de arquitetura

- Arquitetura de software
- Arquitetura de sistemas de informação
- Arquitetura corporativa (enterprise architecture)



# Enterprise Architecture

Uma arquitetura empresarial descreve como **processos de negócios, dados, programas e tecnologias** se unem.

[ASUG 2006, p. 9].

# Information Systems Architecture

Uma **arquitetura de sistemas de informação** refere-se à estrutura ou estruturas do sistema, que compreendem os **elementos do sistema**, as **propriedades externamente visíveis** desses elementos e os **relacionamentos entre eles**.

# Encapsulamento e Abstração

- **Abstração**: simplificar complexidade
- **Encapsulamento**: esconder detalhes internos
- Fundamentos para boa arquitetura

**O que torna uma arquitetura uma "boa"  
arquitetura?**

# Qualidades de uma boa arquitetura

- **Robustez:** tolerância a mudanças
- **Estabilidade:** resistência ao tempo
- **Flexibilidade:** adaptação a novas demandas

# Hierarquia e Abstração – Parnas

- Hierarquia organiza elementos em diferentes níveis
- Abstrações permitem tratar elementos complexos como simples
- Estruturas hierárquicas facilitam manutenção e extensão

# Relações entre Módulos

- Parnas propôs o conceito de '**uses relation**'
- Módulo A usa B se A depende de B para executar corretamente
- Relações bem definidas evitam dependências perigosas

# Múltiplas Estruturas

- Sistemas podem ter diversas estruturas simultâneas
- Visão lógica, física, de implementação, etc.
- Uma estrutura pode dominar, mas outras coexistem



# **Padrões Arquiteturais**

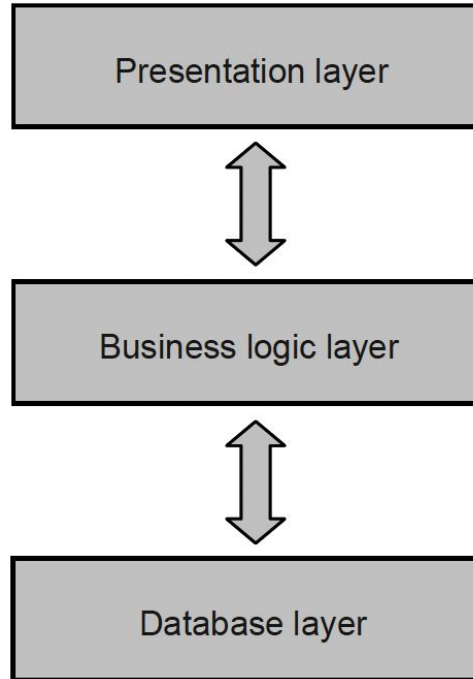
# Padrões Arquiteturais

- Soluções recorrentes para problemas comuns
- Ex: Monolítica, Cliente-Servidor, Camadas, SOA, ESOA
- Facilitam comunicação e padronização entre equipes

# Arquitetura em Camadas

- Separação por responsabilidades
- Interface (apresentação), Lógica (negócios), Dados (armazenamento)
- Torna o sistema modular e manutenível

# Três Camadas Clássicas



# Exemplo

- Um sistema de vendas pode usar arquitetura em camadas
- Separando interface, lógica e banco de dados
- Facilitando alterações futuras sem grandes impactos

# Cliente-Servidor

- Modelo distribuído clássico
- Cliente solicita, servidor responde
- Aplicado em redes locais e na web

# Thin Client vs Fat Client

- **Thin:** processamento no servidor
- **Fat:** processamento no cliente
- Trade-off entre controle e desempenho local

# Rich Clients e AJAX

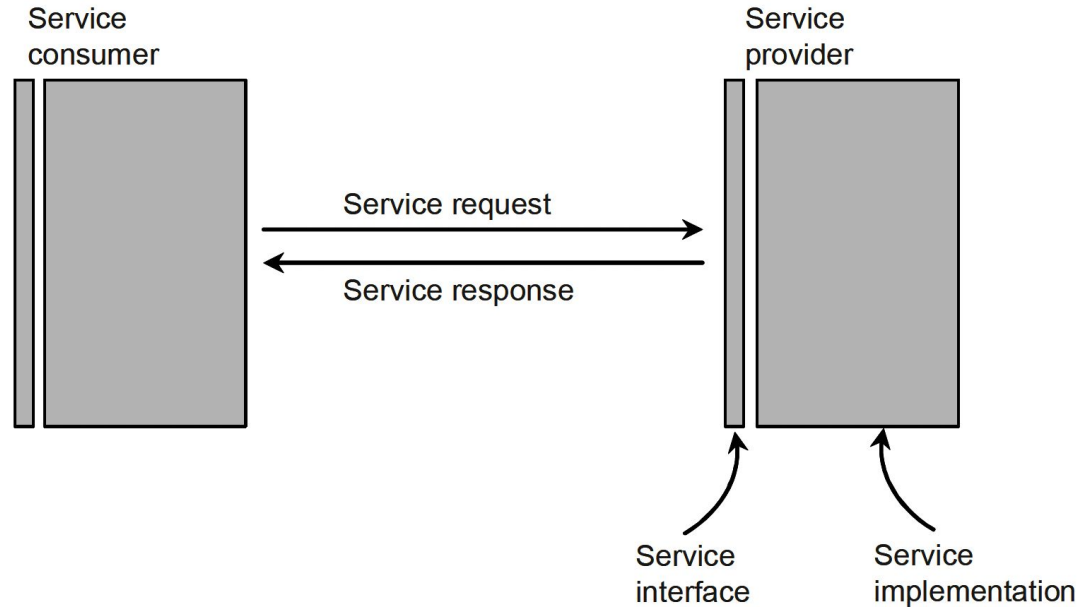
- **Rich Client:** mais lógica no cliente
- Tecnologias como AJAX e RCP permitem respostas rápidas
- Melhora a experiência do usuário



# SOA – Arquitetura Orientada a Serviços

- Sistema visto como um conjunto de serviços
- Maior modularidade, menor acoplamento
- Comunicação por mensagens

# SOA – Arquitetura Orientada a Serviços



# Serviços vs Módulos

- Serviço é uma unidade funcional autônoma
- Interface bem definida (contrato)
- Pode ser executado de forma independente

# SOAP e Mensagens

- SOAP: protocolo baseado em XML
- Define como as mensagens são trocadas
- Comunicação padronizada entre serviços

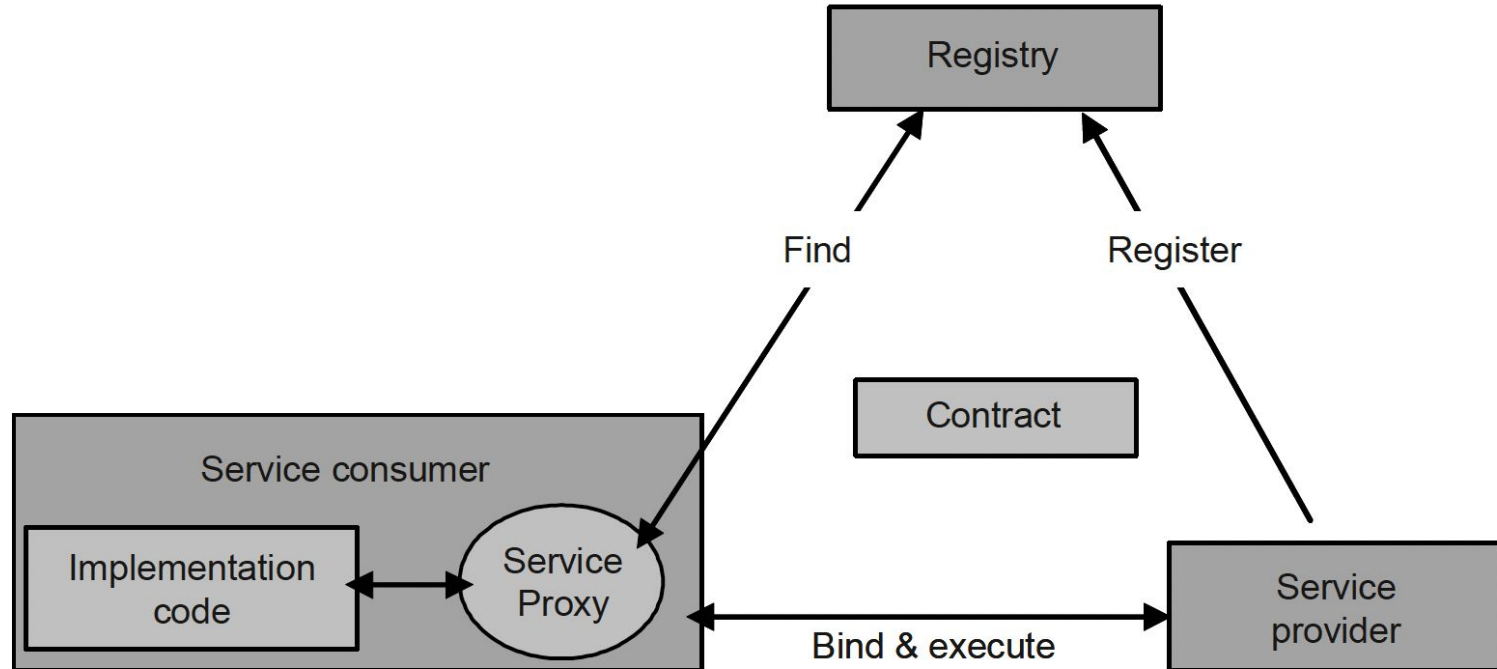
# Web Services

- Permite interoperabilidade entre sistemas distintos
- Publicação via WSDL (Web Services Description Language), descoberta via UDDI (Universal Description, Discovery and Integration)
- Auto-contidos e fracamente acoplados

# Find-Bind-Execute

- Modelo de funcionamento dos serviços
- Encontrar → Vincular → Executar
- Exige contratos claros entre consumidor e provedor

# Find-Bind-Execute

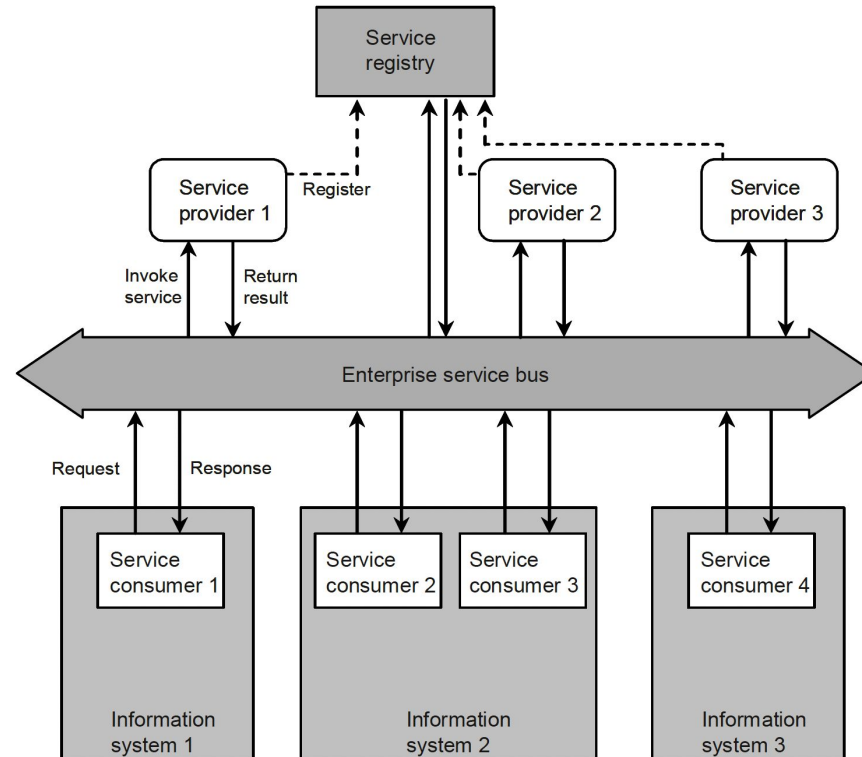


# ESB – Enterprise Service Bus

- Middleware para orquestrar serviços
- Roteamento, transformação, adaptação
- Facilita integração entre diversos sistemas



# ESB – Enterprise Service Bus



# Vantagens do SOA

- Liste três vantagens da arquitetura orientada a serviços
- Compare com modelo monolítico

# ESOA – Enterprise SOA

- Evolução do SOA para o nível de negócios
- Serviços representam processos organizacionais
- Ex: 'Cancelar Pedido', 'Solicitar Compra'

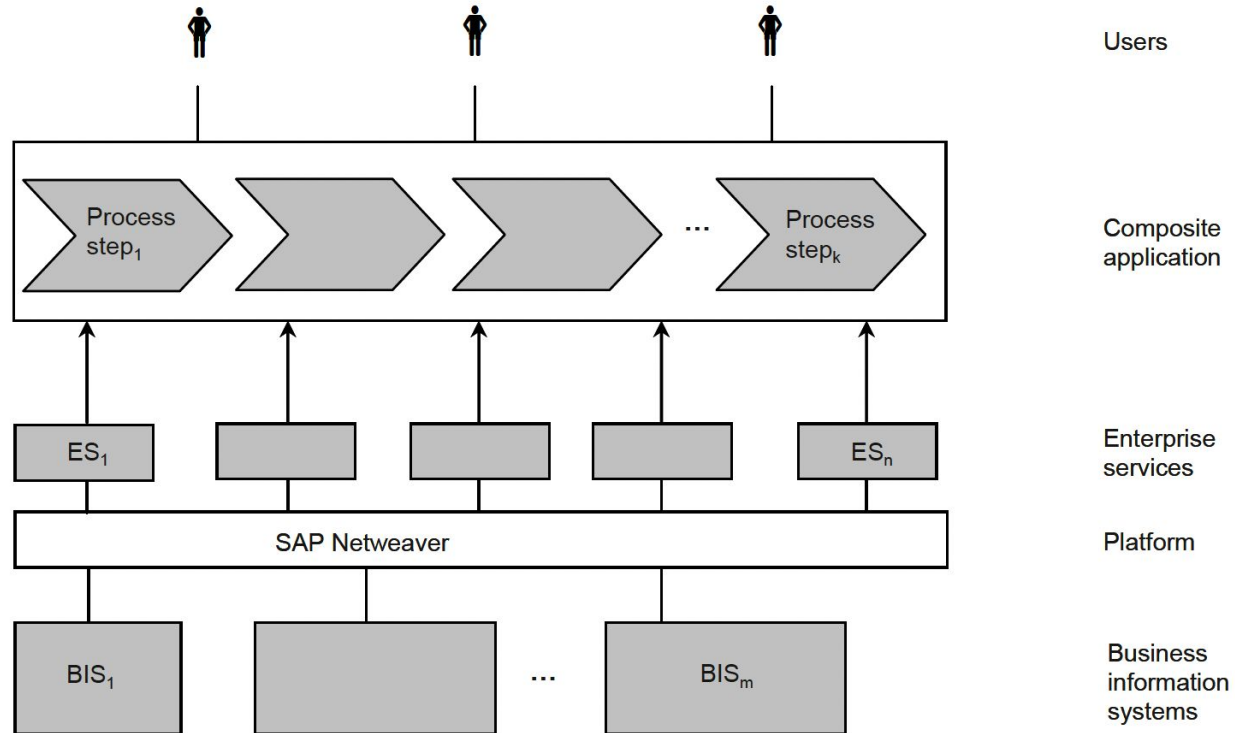
# Enterprise Services

- Serviços compostos por várias operações
- Automação de processos interdepartamentais
- Foco na lógica de negócio

# Composite Applications

- Aplicações compostas de múltiplos serviços
- Permitem inovação rápida sem reescrever sistemas
- Exemplo: solicitação de mudança de produto (PCR)

# ESOA – Enterprise SOA



# Benefícios do ESOA

- Velocidade e flexibilidade na criação de soluções
- Reaproveitamento de sistemas legados
- Abstração dos detalhes técnicos

# Resumo Geral

- Arquitetura: estrutura e relações
- Camadas e serviços ajudam na manutenção
- SOA e ESOA permitem flexibilidade e reuso



# Atividade: Mapear arquitetura

- Qual arquitetura é mais adequada para o sistema que vocês estão projetando?
- ◆ Por que essa arquitetura atende melhor às necessidades do sistema?
  - ◆ Quais seriam as vantagens práticas dessa arquitetura no seu caso?